

UOFIE - Digital Game Development Research

1. Exact Game Idea

Game Name: UOFIE Game

Game Concept

UOFIE is an interactive educational quiz game designed specifically for **students and professors at The French University in Egypt (UFE)**.

The idea is inspired by the concept of Kahoot, where professors can create quizzes based on their lectures and students can join the quiz using their phones or computers.

Instead of simply answering questions, students will earn points, compete with their classmates, and see their position on a leaderboard.

How the Game Works

There will be two main types of users:

1. Professors

Professors will be able to:

- Create an account and log in.
- Create a quiz for a specific course or lecture.
- Add questions and possible answers.
- Select the correct answer.
- Set a time limit for each question.
- Start the game for students.
- View students' results and performance.

2. Students

Students will be able to:

- Log in using their university account.
- Join a quiz using a game code.
- Answer questions within the time limit.
- Earn points for correct and fast answers.
- See their position on the leaderboard.
- View their final scores and results.

The system is inspired by quiz-based platforms such as Kahoot and Quizizz while being designed specifically for UFE's educational environment.

2. What Makes UOFIE Different?

UOFIE will not simply be a copy of Kahoot.

It will be designed specifically for **UFE students, professors, courses, and lectures.**

University Courses

Questions can be organized according to UFE courses.

For example:

- Digital Marketing
- Accounting
- Economics
- Communication
- Management

Lecture-Based Quizzes

Professors can create a quiz for a specific lecture.

For example:

Marketing Lecture 3 - Consumer Behavior

Students can use the quiz to review what they learned directly after the lecture.

University Leaderboards

Think of it as **small competition** → **bigger competition** → **university-wide competition**.

1. Quiz Leaderboard

- This is the simplest one.
- It shows the ranking of students **within one specific quiz**.
- For example, a professor creates:
- **Digital Marketing – Lecture 4 Quiz**
- After the quiz:

Rank	Student	Score
 1	Ahmed	950
 2	Sara	900
 3	Lena	850
4	Omar	800

- This leaderboard would usually be shown immediately after the quiz.
- **Purpose:** Give students immediate competition and show who performed best in that particular quiz.

2. Course Leaderboard

This one is based on a **whole course**, not one quiz.

For example, the Digital Marketing course might have 10 quizzes throughout the semester:

- Lecture 1 Quiz
- Lecture 2 Quiz
- Lecture 3 Quiz
- Lecture 4 Quiz
- etc.

The system can collect students' scores from these quizzes and calculate a **total or average course score**.

For example:

Rank	Student	Course Points
 1	Sara	9,500
 2	Lena	9,200
 3	Ahmed	8,900

- So even if someone didn't come first in every individual quiz, they could still have a high position because of their **consistent performance across the course**.
- **Purpose:** Encourage students to keep participating throughout the semester instead of caring about only one quiz.

3. Class Leaderboard

This compares students **within the same class/group**.

For example, suppose Digital Marketing has:

- Class A
- Class B
- Class C

For **Class A**, the leaderboard could be:

Rank	Student	Points
 1	Student A	8,950
 2	Student B	8,700
 3	Student C	8,450

This is useful when a course has multiple classes or sections.

You could also eventually have a **class-vs-class competition**, for example:

- Class A: 85,400 points
- Class B: 82,900 points
- Class C: 79,600 points
- That would make the game more social because students aren't only competing individually.

4. Department Leaderboard

This would be a bigger level of competition.

Instead of comparing students from one class, the system could compare students **within an entire department**.

For example, in the **Digital Marketing/Business-related department**, students from different courses and classes could contribute points.

- The leaderboard might look like:

Rank	Student	Total Points
 1	Student A	18,500
 2	Student B	17,900
 3	Student C	17,200

- This would work across multiple courses rather than only one course.
- **Purpose:** Create a larger university community around the game.

How They Connect Together

One Quiz → One Course → One Class → One Department

So, the student's achievements can grow from a small competition into a larger one.

Important thing for this project

- **I don't have to build all four leaderboards in the first version.**
- For My MVP, I recommend starting with:

Quiz Leaderboard

Then, after the basic system works, you can add:

Course → Class → Department

Professor Dashboard

Professors could see:

- Number of participating students
- Average score
- Most difficult questions
- Percentage of correct and incorrect answers
- Student performance

Gamification Features

Students could receive:

- Points
- Badges
- Levels
- Streaks
- Leaderboard positions

For example:

 **Quiz Master** – Score above 90%

 **3-Day Streak** – Participate in quizzes three days in a row

 **Top 3** – Finish among the top three students

These features can make revision more interactive and encourage students to participate.

3. How Could the Game Be Developed?

For the first version, I would develop UOFIE as a **web-based game**.

Students would be able to access it from their phones, tablets, or computers without installing an application.

The basic structure would be:

HTML + CSS + JavaScript → PHP → MySQL

Why a Web-Based Game?

A web-based game would be practical for a university because:

- Students can use their phones or computers.
- No application installation is required.
- Professors can access the system from their computers.
- Updates can be made centrally.
- The system can later be connected to UFE university systems.

4. Programming Languages

HTML

HTML will be used to create the basic structure of the website.

It can be used for:

- Login page
- Student dashboard
- Professor dashboard
- Quiz page
- Leaderboard
- Results page

CSS

CSS will be used to control the visual appearance of the game.

It will be used for:

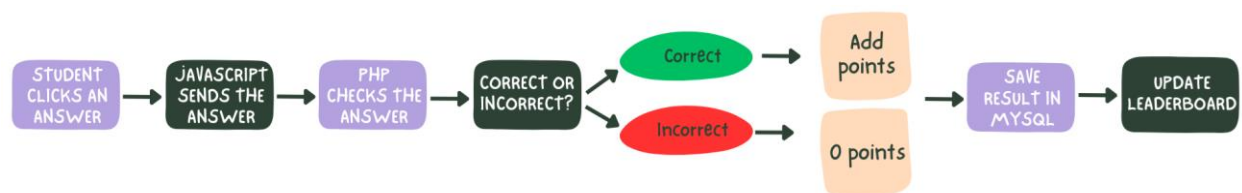
- Colors
- Fonts
- Buttons
- Layout

- Responsive design
- Animations
- Spacing and positioning

JavaScript

JavaScript will control the interactive parts of the game.

For example:



JavaScript can also control:

- Countdown timers
- Interactive buttons
- Answer selection
- Showing feedback
- Sending information to PHP
- Updating parts of the page without reloading

PHP

PHP will be the **backend programming language**.

It will communicate between the front-end and the MySQL database.

PHP will be responsible for:

- User registration and login
- Authentication
- Creating quizzes
- Saving questions

- Checking answers
- Calculating scores
- Saving results
- Generating game codes
- Generate the games QR codes
- Retrieving quiz information
- Communicating with MySQL

5. Database

The game will need a database because information must be stored and retrieved.

The database I plan to use is **MySQL**.

The project can be developed using **XAMPP**, which provides Apache and MySQL for local development.

The database could contain tables such as:

Students

Student ID	Name	Email	Department
------------	------	-------	------------

Professors

Professor ID	Name	Email	Department
--------------	------	-------	------------

Courses

Course ID	Course Name	Professor
-----------	-------------	-----------

Quizzes

Quiz ID	Quiz Title	Course	Lecture	Professor
---------	------------	--------	---------	-----------

Questions

Question ID	Quiz ID	Question	Answer 1	Answer 2	Answer 3	Answer 4	Correct Answer
-------------	---------	----------	----------	----------	----------	----------	----------------

Results

Result ID	Student	Quiz	Score	Correct Answers	Time
-----------	---------	------	-------	-----------------	------

These tables would allow the system to store users, courses, quizzes, questions, and student results.

6. How PHP Would Work in the Game

When a professor creates a quiz



The professor fills in the quiz information and questions.

PHP receives the data and saves it inside MySQL.

When a student enters a game code



The student enters the code provided by the professor.

PHP checks the database to find the corresponding quiz and allows the student to join.

When the student answers a question



7. What PHP Can Handle

The PHP backend can handle:

- Student registration and login
- Professor accounts
- Quiz creation
- Saving questions
- Authentication
- Joining a game using a game code
- Calculating scores
- Saving leaderboard results
- Generating professor reports
- Communicating with MySQL

8. Real-Time Multiplayer System

One important feature of UOFIE Game is allowing multiple students to participate in the **same quiz at the same time**.

For example, the professor creates:

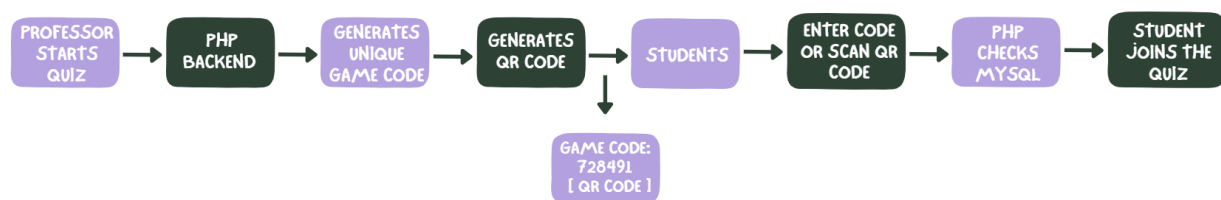
Marketing Quiz – Lecture 4

When the professor starts the quiz, the system generates a unique:

GAME CODE: 728491

The system also generates a **QR code** containing the game joining link. Students can either enter the game code manually or scan the QR code with their phones to join the quiz.

How Students Join the Game



The QR code can contain a joining URL such as:

<https://UOFIE.com/join.php?code=728491>

When a student scans the QR code, their phone opens the joining page and the PHP backend checks whether the game code is valid and active.

How the Quiz Works After Students Join



For the first version, communication between the students' browsers and the PHP server can be handled using **JavaScript with AJAX or Fetch requests**. This allows information to be sent to PHP without requiring the entire page to reload.

For a future, more advanced version, **WebSockets** could be added to provide faster two-way communication between the server and all connected students.

Therefore, **Node.js is not required for the first version**. The project can be developed using:

HTML + CSS + JavaScript → PHP → MySQL

with **QR codes** providing an easy way for students to join a game.

9. Tools and Software Needed

Visual Studio Code

Visual Studio Code can be used as the main code editor for:

- HTML
- CSS
- JavaScript
- PHP
- SQL

XAMPP

XAMPP can be used for local development.

It provides:

- Apache web server
- MySQL database
- PHP environment
- phpMyAdmin for managing the database

phpMyAdmin

phpMyAdmin can be used to:

- Create the database
- Create tables

- Insert data
- Modify data
- View database records
- Manage MySQL

GitHub

GitHub can be used to:

- Store the project
- Track changes
- Keep previous versions
- Collaborate with team members
- Upload the source code

10. Graphics and UI/UX Design

The game needs a simple and attractive design because it is intended for university students.

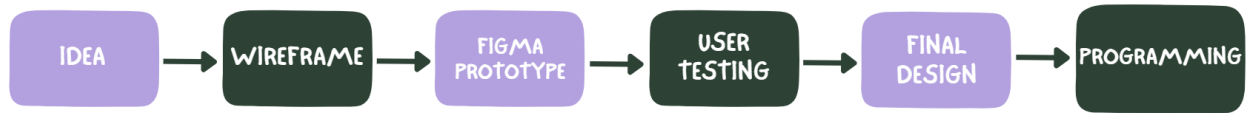
Figma

Figma can be used to create the UI/UX design before programming.

I could design:

- Welcome page
- Login screen
- Student dashboard
- Professor dashboard
- Quiz screen
- Answer buttons
- Leaderboard
- Results page
- Student profile
- Badges

The design process could be:



Canva

Canva can be used to create:

- Game logo
- Icons
- Background elements
- Presentation graphics
- Promotional designs

11. Animations

Animations can make the game feel more interactive and enjoyable.

For example:

Correct Answer



Correct! +100 points

A small animation can appear after the student selects the correct answer.

Wrong Answer

Oops! Try again on the next question!

Leaderboard

The leaderboard can have simple animations when students move up or down.

For example:

	Student A	950
	Student B	900 ↑
	Student C	850

Possible tools include:

- CSS animations
- JavaScript animations
- Lottie animations
- Figma prototypes

12. Sound

Sound effects can make the game more engaging.

Examples include:

 Countdown sound

 Correct-answer sound

 Wrong-answer sound

 Winning sound

 Level-up sound

Possible tools:

- **Audacity** for editing sounds
- Sound-effect libraries for finding suitable effects
- **JavaScript** for playing sounds inside the game

The game should also include a **mute option** so students can turn the sounds off.

13. Video

Videos could be included in some questions when professors want to explain a topic visually.

For example:

Watch this 20-second video and answer the question.

Videos could be:

- Uploaded by professors
- Embedded from approved sources
- Used as part of a question

Possible tools include:

- Canva
- CapCut
- Adobe Premiere Pro

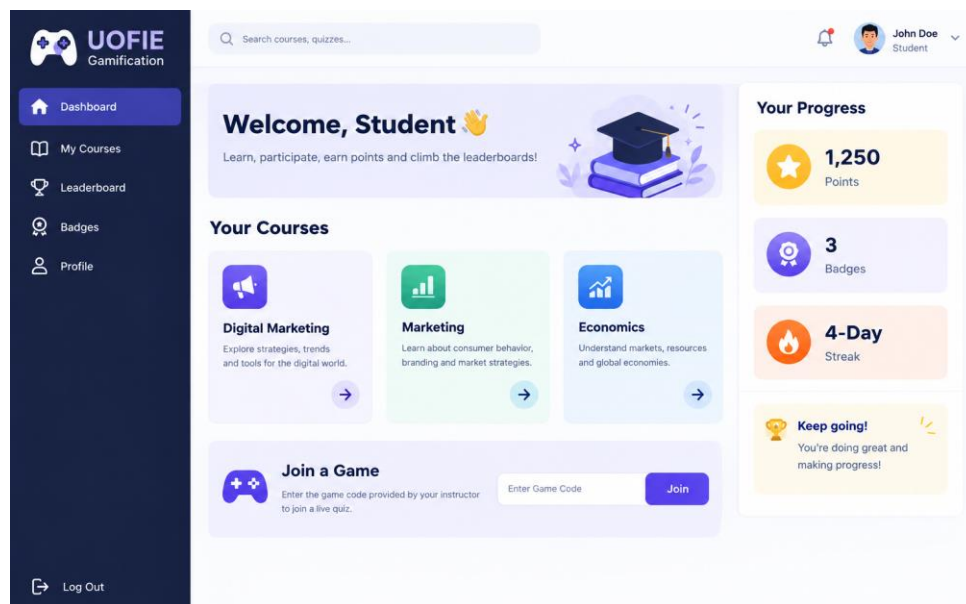
14. Main Screens of the Game

The first version could contain the following screens.

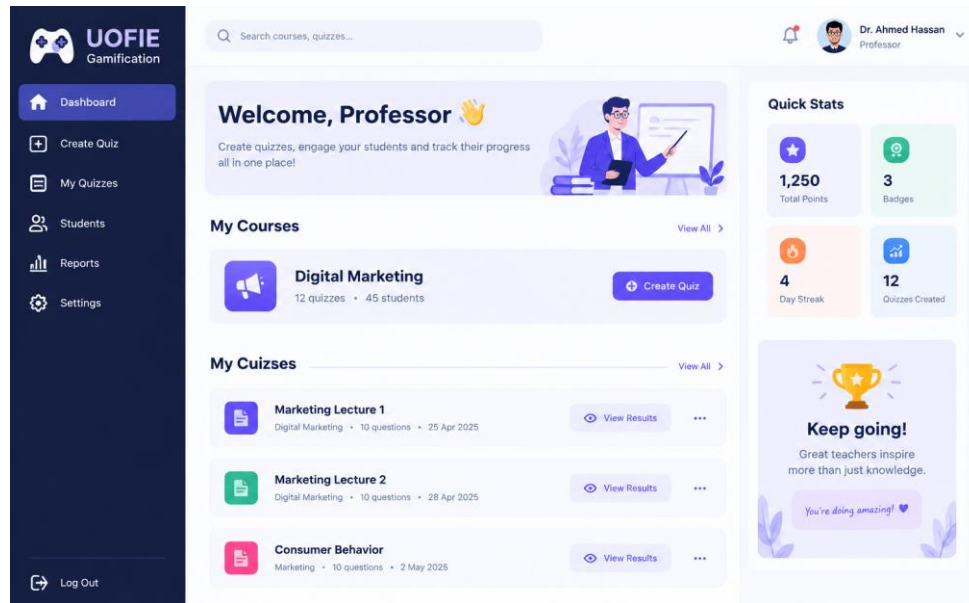
1. Welcome Page



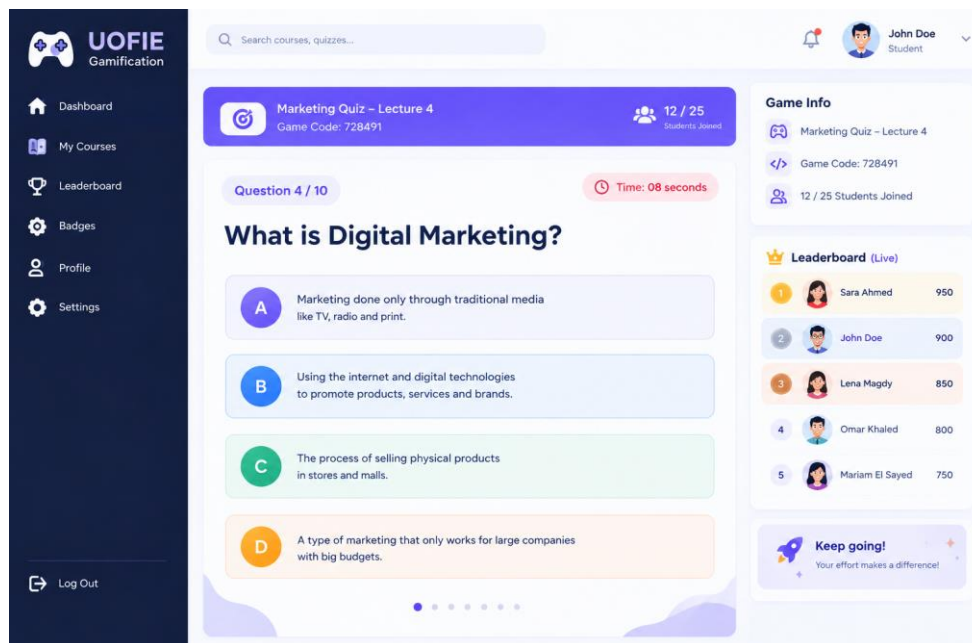
2. Student Dashboard



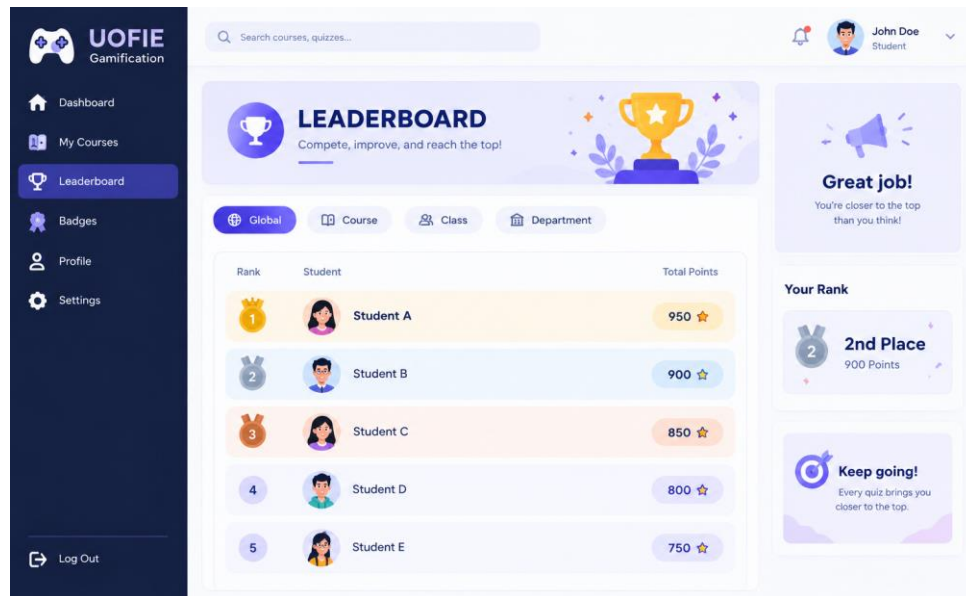
3. Professor Dashboard



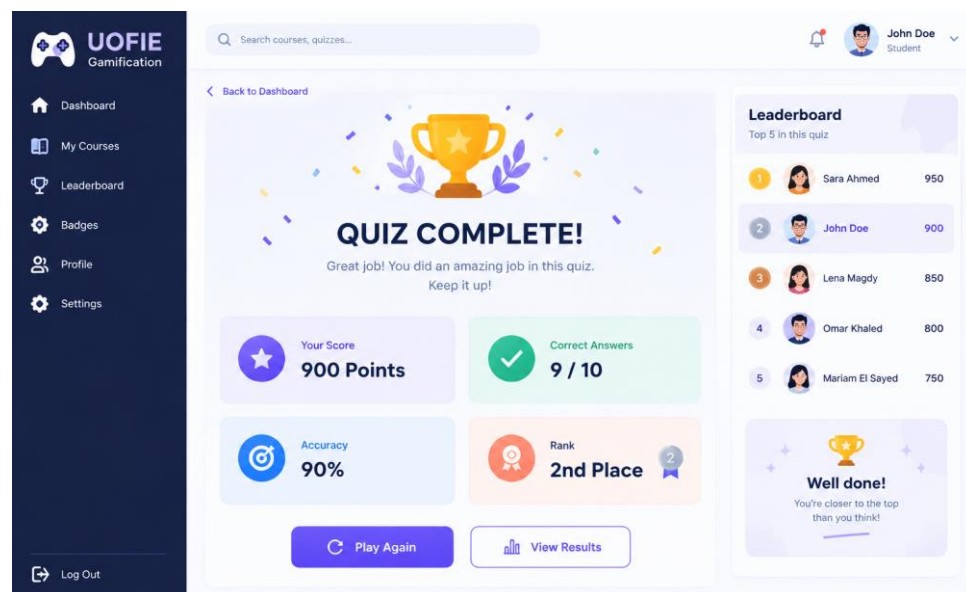
4. Quiz Screen



5. Leaderboard



6. Final Results



15. Development Process

The project can be divided into several stages.

Stage 1 - Research

Research:

- Kahoot
- Gamification
- Educational games
- Quiz platforms
- UFE students' needs
- Different types of online quiz systems

Stage 2 - Planning

Decide:

- Game rules
- Users
- Features
- Scoring system
- Question types
- Database structure
- Number of screens
- How students will join a game

Stage 3 - UI/UX Design

Create the screens and prototype using Figma / See inspirations from [Awwwards.com](https://www.awwwards.com).

Stage 4 - Database Development

Create the MySQL database and tables.

For example:

Students

Professors

Courses

Quizzes

Questions

Results

Stage 5 - Prototype

Create a small working version containing:

- Login
- Join game
- Questions
- Timer
- Score
- Leaderboard

Stage 6 - Programming

Develop:

- Front-end
- PHP backend
- MySQL database
- Game logic
- Score calculation
- Communication between JavaScript and PHP

Stage 7 - Testing

Test the system with a small group of UFE students.

Check:

- Are questions displayed correctly?
- Does the timer work correctly?
- Are scores calculated correctly?
- Does the leaderboard update?
- Can multiple students join?
- Does it work on mobile?
- Does the PHP backend save data correctly?
- Does the database store the results correctly?

Stage 8 - Improvements

Use student and professor feedback to improve the game.

Stage 9 - Launch

Deploy the web application and make it available to UFE students and professors.

16. Minimum Viable Product (MVP)

The first version should focus on the most important features instead of trying to build everything at once.

The MVP could contain:

- ✓ Student login
- ✓ Professor login
- ✓ Create quiz

- ✓ Join quiz using a game code
- ✓ Multiple-choice questions
- ✓ Timer
- ✓ Automatic scoring
- ✓ Leaderboard
- ✓ Final results
- ✓ MySQL database
- ✓ PHP backend

Future versions could add:

- ★ Badges
- 🔥 Streaks
- 📊 Advanced analytics
- 🎥 Video questions
- 🎵 Sound effects
- 🏆 University-wide competitions
- 📱 Mobile application
- 🔑 UFE account integration

17. Final Technology Stack

Category	Tool
Front-end	HTML, CSS, JavaScript
UI/UX Design	Figma
Backend	PHP
Database	MySQL
Server	Apache / XAMPP
Database Management	phpMyAdmin
Graphics	Canva / Figma
Animations	CSS / JavaScript
Sound	Audacity
Video	Canva / CapCut
Code Editor	Visual Studio Code
Version Control	GitHub
Real-Time Communication	AJAX / Fetch / Polling
Future Real-Time Option	WebSockets

18. Why This Game Is Useful for UFE

The main purpose of UOFIE Game is not only entertainment.

It aims to make studying and revision more interactive by combining **education with game mechanics**.

Benefits for Professors

Professors could:

- Make revision more interactive.
- Check students' understanding quickly.
- Identify difficult topics.
- View student performance and results.

Benefits for Students

Students could:

- Review lectures in an enjoyable way.
- Receive immediate feedback.
- Participate in classroom activities.
- Compete with classmates.
- Stay more engaged during revision.

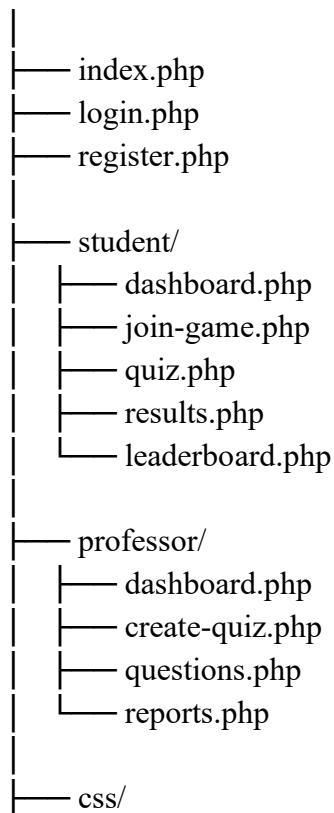
The basic idea is:

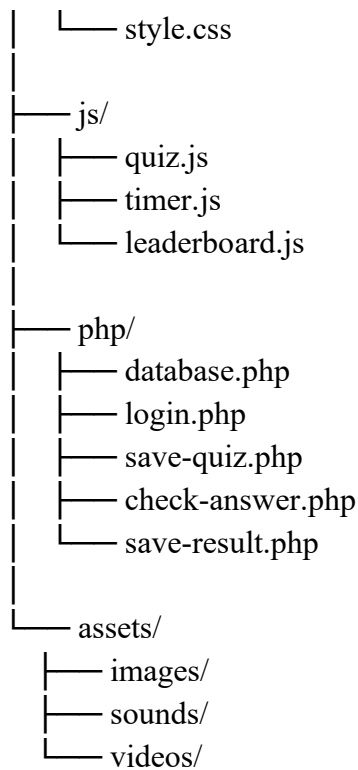
Traditional Lecture → Quiz → Game → Competition → Feedback → Better Engagement

19. Project Structure

A possible project structure could be:

UOFIE-Game





This structure separates the student side, professor side, PHP backend, JavaScript, and media files.

20. Short Conclusion

UFE Gamification is a university-based educational quiz game inspired by the concept of Kahoot. The platform will allow UFE professors to create quizzes based on their lectures and allow students to participate using a game code.

The first version will be developed as a web-based application using **HTML, CSS, JavaScript, PHP, MySQL, and Apache through XAMPP**. Figma and Canva will be used for the interface and visual elements, while CSS and JavaScript can be used for animations. Audacity can be used for sound editing, and Canva or CapCut can be used for video content.

The game will include features such as timed questions, automatic scoring, leaderboards, student accounts, professor dashboards, and quiz results.

Future versions could include badges, streaks, advanced analytics, video questions, university-wide competitions, and integration with UFE student accounts.

The goal is to create a simple, interactive, and university-specific learning platform that combines education and gamification to make lecture revision more engaging for UFE students.

21. Final Description of the Idea

UFE Gamification is a university-specific interactive learning platform inspired by quiz-based gamification. Professors can create quizzes based on their lectures, while students can join using a game code, answer questions within a time limit, earn points, and view their results on a leaderboard. The system will be developed as a web application using HTML, CSS, JavaScript, PHP, and MySQL, with Apache/XAMPP used for the development server.